Adaptive Multi-Environment Driving Cycle Construction Using GPS–OBD Data

Pipeline documentation: what each stage does, how it is implemented, why it was designed this way, and the evidence that it works

SLIIT Final-Year Research Component · IT23278080 · Electric vehicle (BYD Atto 3), Car Scanner + ELM327 logs

Important – data status. Every number in this document comes from running the notebooks on the **synthetic** test dataset (77 generated Car Scanner logs on two routes: Panadura–Wellawatte urban and Gelanigama–Kadawatha expressway). The results show that the **method works and is verifiable**; they are **not** findings about real Sri Lankan EV driving. When real trips are placed in `data/raw`, Stages 1–11 are re-run unchanged and the real results replace these.

Pipeline at a glance

Stage	Notebook	In one sentence	Main output
1	01_Raw_Data_Inventory	Look at every raw file and report what data exists and how good it is.	inventory.csv
2	02_Data_Cleaning	Remove impossible values, duplicates and GPS errors; record gaps.	cleaned trips
3	03_GPS_OBD_Synchronization	Put GPS and OBD on one 1-second timeline.	1 Hz trips
4	04_Map_Matching	Find the road the EV was on every second.	road class per second
5	05_Environment_Labelling	Label every second Urban / Suburban / Intercity / Rural / Expressway.	labelled trips
6	06_Adaptive_Segmentation	Cut trips into stop-to-stop pieces of one environment.	1,073 segments
7	07_Feature_Extraction	Describe each segment with speed, acceleration, stop features.	feature table
8	08_KMeans_Clustering	Group similar segments inside each environment.	clusters
9	09_Representative_Selection	Pick real segments that represent each environment.	selected traces
10	10_Driving_Cycle_Construction	Join them into one speed–time cycle per environment.	cycle CSVs
11	11_Statistical_Validation	Test the cycles against trips never used to build them.	validation report

Each notebook follows the same pattern: **paths → settings (CFG) → functions → run → tables and figures → automatic verification table → key results**. Raw files are never modified (Stage 2 proves this with MD5 fingerprints) and every stage writes to its own folder, so any result can be traced back to its source.

Stage 1 – Raw data inventory

Before processing anything, open every raw file and write down what is in it: how many rows, which signals, how often they are recorded, and what problems can already be seen.

What the code does

- Finds every Car Scanner CSV in `data/raw` and gives it a trip ID (T001–T077) in date order.
- Reads each file (53 columns: 28 OBD-II parameters and 23 phone / app-calculated columns) and measures rows, duration, reading interval per signal, GPS fixes, gaps, frozen GPS, GPS jumps, duplicates, invalid values and coordinates outside Sri Lanka.
- Adds a *route hint* from the median GPS longitude (Route A / Route B) – used only for reporting, never as an environment label.
- Writes `inventory.csv` and a parameter-availability table.

Why

- The methodology requires inspecting real files before choosing any processing method – nothing is assumed about the schema or which EV signals exist.
- The inventory showed the key fact for the whole pipeline: **OBD speed arrives only every ~3.7 s, GPS every 1 s**. This decided the synchronisation method in Stage 3.

Evidence

Item	Result
Files / rows	77 files, 642,588 rows, 47.7 h, 1,933.6 km (3 Aug – 12 Oct 2026)
Route A (urban)	38 trips, 28.4 h, 586.9 km, max 54.6 km/h
Route B (expressway)	39 trips, 19.3 h, 1,346.7 km, max 101 km/h
Sampling	OBD speed / battery every 3.7 s; GPS position every 1.0 s

Fix made during review: the notebook first saved to a relative path (Colab's temporary disk), so `inventory.csv` never reached Google Drive and later stages showed only “all trips”. Paths were changed to the Drive folder; route-level reporting now works in Stages 4–5.

Stage 2 – Data cleaning

Remove values that are physically impossible or are recording errors, while keeping every genuine driving event (hard braking, real stops). Every change is logged, and gaps are recorded – not filled.

What the code does

- **Column decisions:** 52 raw columns → 9 kept (speed, motor rpm, battery current and voltage, SoC, GPS lat/lon, GPS speed, altitude). Every drop has a written reason (duplicate column, constant, app-calculated, start-up only).
- **Cleaning steps:** exact duplicate rows; values outside physical limits set to missing; repeated GPS positions (Car Scanner copies the last fix into every row); GPS jumps (> 25 m from the median track of 11 neighbours); speed spikes (up AND down faster than 5 m/s²); idle time before departure / after arrival trimmed.
- **Gaps recorded for Stage 3:** logging gaps (> 60 s no rows), OBD dropouts (> 12 s), GPS gaps while moving (> 30 s).
- MD5 fingerprints prove the raw files were not modified.

Why these limits

Limit	Value	Physical basis (BYD Atto 3)
Pack voltage	315–460 V	126 cells × 2.5 V (empty) … 126 × 3.65 V (full) = 315 / 459.9 V
Battery current	± 500 A	150 kW ÷ 315 V = 476 A at the lowest voltage
Motor rpm	13,000	160 km/h × 78 rpm per km/h = 12,480
Speed	200 km/h	top speed 160; 255 = the 0xFF 'no answer' code
Spike acceleration	5 m/s²	0–100 km/h in 7.3 s = 3.8 m/s² average
GPS jump	25 m	phone GPS error ≈ 3–5 m; 25 m is ≥ 5× larger

Evidence

- **Rows:** 642,588 → 322,910 (most removed rows were empty after dropping unused columns). 34 duplicates, 4 invalid values each for speed / voltage / current, 86 GPS jumps in 13 trips, 2.0 h idle trimmed. 8/8 checks PASS.
- **Margins:** clean speed max 101 km/h vs limit 200; current ±166 A vs ±500; voltage 400–431 V inside 315–460 – limits remove only impossible values.
- **GPS threshold is not critical:** 99.9 % of fixes lie within 9.7 m of the track. Every threshold from 15 to 25 m flags exactly the same 13 trips; 10 m starts flagging noise (23 trips), ≥ 35 m misses small jumps.
- **Injected faults detected** (known from the synthetic manifest): duplicates 12/12, GPS freeze 13/13, OBD dropout 13/13, late start 7/7, early end 2/2, invalid values 8/9 (one was in a dropped column), GPS jumps 13/14, app-suspended 3/4.

Limitation: with speed every 3.7 s, a one-reading spike is only detectable above ≈ 5 × 3.7 × 3.6 ≈ 67 km/h. No spikes were injected, so this rule is tested only by the spike-test cell.

Stage 3 – GPS–OBD synchronisation

GPS and OBD readings arrive at different moments. This stage puts every signal on one shared 1-second timeline, marks each value as observed / interpolated / missing, and never fills long gaps.

What the code does

- Builds a 1 Hz grid per logging segment (never across a logging gap).
- **Speed:** PCHIP interpolation (shape-preserving: no overshoot, no negative speeds) across gaps ≤ 8 s (≈ 2 polling cycles). **Battery, rpm:** linear ≤ 8 s. **GPS:** linear ≤ 5 s. **SoC:** held ≤ 30 s.
- Searches for a clock offset between OBD and GPS (±5 s) by correlating OBD speed with GPS-derived speed; applies it only if correlation ≥ 0.8.
- Measures the speedometer ratio (OBD ÷ GPS speed).

Why

- Driving cycles (e.g. WLTP) are defined at 1 Hz, and acceleration needs a regular time step.
- Gap limits follow the methodology rule “do not interpolate across long gaps”: 8 s covers one missing reading, but no OBD dropout (all > 12 s) is ever filled.

Evidence

Test	Result
Output	163,461 seconds (45.4 h); speed 26.2 % observed, 72.8 % interpolated, 1.0 % missing
Speedometer check	OBD reads **+1.5 %** above GPS – exactly the offset set in the synthetic generator
Lag detection	Injected offsets −4 … +5 s recovered exactly; +7 s (outside search range) correctly not found
Interpolation accuracy	vs independent GPS speed: interpolated MAE 0.55 km/h (p95 1.83), observed 0.38 (p95 1.17), bias −0.02
Dropout check	1,141 seconds inside OBD dropouts tested – none filled
Verification	8/8 PASS (1 s steps, distance preserved within 1 %, $\|a\| \leq 5$ m/s², …)

Two bugs were found and fixed during review: the dropout check compared against gap times parsed as dates (so it tested nothing), and GPS speed was saved empty (max_gap = 0). Limitation: 73 % interpolated speed smooths short acceleration peaks.

Stage 4 – GPS map matching

GPS points are noisy (a few metres off the road). Map matching decides which real road the EV was driving on every second. The road type (motorway, trunk, residential …) is the most important evidence for the environment label in Stage 5.

What the code does

- **Study area:** GPS points of all trips, buffered by 600 m (one point per 100 m cell).
- **Road network:** OpenStreetMap drivable roads downloaded once with OSMnx and saved (`drive_network.graphml`), plus traffic signals and toll booths.
- **Candidates:** for every GPS point, up to 8 road segments within 35 m.
- **Hidden Markov Model + Viterbi (Newson & Krumm, 2009):** chooses the most likely *sequence* of roads. Two probabilities are combined: (1) *emission* – how close the point is to the road (GPS error σ = 5 m); (2) *transition* – whether the driving distance along the network between two consecutive roads agrees with the straight-line distance the GPS moved (β = 10 m). One-way rules and connectivity are respected.
- Attaches road class, name, speed limit, lanes, distance to the nearest junction, near-signal and near-toll flags, and a match flag: matched / low_confidence / unmatched / no_gps.

Why a Hidden Markov Model, not “nearest road”

Approach	Problem	Effect on this research
Nearest road to each point	Ignores how roads connect; jumps between parallel roads, service roads and ramps	Expressway seconds wrongly labelled as the road beside it → wrong environment
HMM map matching (used)	Considers the whole path: a car cannot jump to an unconnected road in 1 s	Correct road class even next to parallel roads, at interchanges and on ramps

Why OpenStreetMap

- Free and open (ODbL licence), covers Sri Lanka, and contains exactly the evidence needed: road class (`highway` tag), junction topology, one-way rules, signals and toll booths.
- Reproducible: the network is downloaded once, saved with its download date and settings in `download_info.json`, and reused by Stage 5.

Problem found and fixed – the strongest evidence that the checks work

The first run left **58.4 % of all seconds unmatched**. Diagnosis showed the urban route had matched 0 %. Cause: `retain_all=False` keeps only the largest connected road network. The two routes are ~30 km apart, so OSMnx kept the expressway network and silently deleted the whole Galle Road network.

Measure	Before (retain_all=False, 300 m)	After (retain_all=True, 600 m)
Study area	34.5 km²	68.5 km²
Road network	765 nodes, 458 junctions	6,710 nodes, 4,034 junctions
Trunk road length (Galle Road A2)	2.9 km	22.3 km
Unmatched seconds	58.4 %	**0.0 %**
“≥ 95 % of GPS matched” check	39/77 trips	**77/77 trips**
Signals / toll booths	40 / 22	45 / 24

The buffer was raised to 600 m because Stage 5 counts junctions within 500 m of the car – the map must reach at least that far, or junction density would be undercounted near the route.

Evidence of match quality

- GPS-to-road distance: **median 2.1 m, 95th percentile 5.9 m** – within normal phone GPS error.
- Road-class share of matched time: trunk 43 %, motorway 31.7 %, primary 18.4 %, motorway link 5.4 % – consistent with Galle Road (A2, trunk) and the E01/E02 expressway.
- Verification: 5 of 6 checks PASS (same rows as Stage 3, no road without GPS, all matches within 35 m, ≥ 95 % matched, no > 100 km/h on residential roads).

The one failing check – explained, not hidden

“No impossible jumps” passed for 53/77 trips. A heading check (northbound vs southbound) located the cause exactly:

Route	Direction	Trips	Trips with jumps	Median distance to road
Expressway	northbound (forward)	30	1	1.6 m
Expressway	**southbound (reverse)**	9	**9**	11.7 m
Urban	northbound (forward)	24	0	2.1 m
Urban	**southbound (reverse)**	14	**14**	4.9 m

Every reverse trip – and essentially no forward trip – has the problem, and the counts match the synthetic manifest exactly. The generator placed reverse-trip GPS on the opposite carriageway of dual roads; the matcher correctly refused to drive the wrong way on a one-way carriageway and snapped 5–12 m across. **The road class (and therefore the environment label) is unaffected**, because both carriageways have the same class. This is a synthetic-data artefact that will not occur with real GPS.

Limitations: OSM completeness in Sri Lanka varies; parameters (35 m, σ = 5 m, β = 10 m) are typical values for phone GPS and were not tuned; signals/toll booths are as mapped in OSM.

Stage 5 – Driving environment labelling

Give every second one of five labels – Urban, Suburban, Intercity, Rural, Expressway – or Unknown, using evidence about the **road and its surroundings**, never the vehicle's speed.

What the code does

Rule	Label	Condition	Meaning
1	Expressway	matched road is motorway or motorway_link	access-controlled E01/E02 incl. ramps and toll plazas
2	Urban	not expressway, ≥ 50 junctions/km² within 500 m	dense street grid = built-up city
3	Suburban	15–50 junctions/km²	moderate grid = city fringe
4	Intercity	< 15 junctions/km², trunk / primary / secondary road	main road through open land
5	Rural	< 15 junctions/km², minor road	village road through open land
6	Unknown	no matched road	no evidence → no guess

- **Junction density:** number of OSM junctions (≥ 3 roads meeting) within 500 m ÷ 0.785 km².
- **Smoothing:** short Unknown gaps (≤ 10 s) filled when the same label is on both sides; label runs shorter than 30 s merged into a neighbour (never across a logging gap). Changed only 0.6 % of seconds.
- **Confidence** per second: high / medium (density near a threshold, uncertain map match) / low (label set by smoothing) / none (Unknown). Built-up land use from OSM is used as *supporting* evidence only.
- Every second stores its evidence (road class, junction density, built-up share) and status “rule-based (unverified)”.

Why these evidence sources

Option	Used?	Reason
Vehicle speed	**No**	The cycles are built from speed; labelling by speed would be circular (the methodology forbids it) and would label a traffic jam on the expressway as “Urban”.
OSM road class	Yes	Directly identifies the expressway network and main vs minor roads; objective and reproducible.
Junction density	Yes	A dense street grid is a measurable sign of built-up area; available everywhere from the same OSM network; does not depend on the car's behaviour.
Land use polygons	Support only	Sparse in Sri Lankan OSM (48 polygons in 68.5 km²): a missing polygon means “not mapped”, not “open land”.
Population / administrative data	No	Not available at road level for these routes in a reproducible form.

Evidence

1. The labels agree with the known routes (route hint from Stage 1, independent of the rules):

	Expressway	Urban	Suburban	Unknown
Route A – Panadura–Wellawatte (urban)	0.0 %	**91.1 %**	6.2 %	2.7 %
Route B – Gelanigama–Kadawatha (expressway)	**91.6 %**	4.8 %	2.0 %	1.6 %

The ~7 % non-expressway time on Route B is the local road at each end before and after the expressway – correct behaviour, not an error.

2. The Stage 4 fix had exactly the expected effect: Unknown fell from **60.7 % → 2.3 %** (the remaining 2.3 % are seconds without GPS), Urban + Suburban rose from 2.7 % → 61.1 %, Expressway stayed at 36.6 %.

3. Sensitivity of the thresholds (both thresholds changed by ±25 %):

Thresholds	Urban	Suburban	Expressway
× 0.75 (38 / 11 per km²)	59.5 %	1.9 %	36.2 %
× 1.00 (50 / 15 per km²)	56.7 %	4.7 %	36.2 %
× 1.25 (62 / 19 per km²)	50.6 %	10.8 %	36.2 %

Expressway is unaffected (road-class rule). Urban changes moderately; the Urban/Suburban split is sensitive – reported openly as a limitation.

4. Verification: 6/6 checks PASS – same rows as Stage 4, every second labelled with a confidence, labels only from the allowed set, Expressway only on motorway/ramp, no label run shorter than 30 s, evidence stored for every second.

Limitations (state in the viva). (1) Labels are not yet checked against independent ground truth: the synthetic truth files were not on Drive; a ready validation cell computes accuracy and a confusion matrix once they are uploaded. (2) The 50 / 15 junctions/km² thresholds are judgement-based – cite literature and recalibrate on real, hand-checked trips. (3) No Intercity or Rural driving exists in this dataset – documented, not fabricated. (4) In the current run, missing land use lowered Urban “high confidence” to 6.8 %; a revised rule (land use only adds doubt where it is actually mapped) is recommended.

Stage 6 – Adaptive segmentation

Cut every trip into short pieces (segments) that each belong to one environment and normally run from one stop to the next. These are the building blocks of the driving cycles.

What the code does

- Splits trips into **blocks** at logging gaps, missing speed and environment changes.
- Inside a block, starts a new segment at every real **stop** (speed < 2 km/h for ≥ 3 s) → stop-to-stop micro-trips.
- Merges **queue creeping** (< 10 s moving or never reaching 10 km/h) into the next micro-trip.
- Cuts stop-free stretches longer than 15 min (expressway cruising) into equal parts at the **steadiest** second, so no acceleration or braking is split.
- Flags each segment: **good** (starts and ends at rest or at a steady-cruise cut), **partial** (cut while moving, e.g. at an environment change), **exclude** (Unknown, < 20 s, no movement).

Why – four methods compared on the same data

Method	Segments	Mixed environments	Cuts at rest	Cuts during accel./braking
Fixed 120 s windows	1,401	14.8 %	9.7 %	19.1 %
Plain stop-to-stop	848	21.2 %	100 %	0.0 %
Change-point detection	1,405	13.5 %	1.2 %	78.2 %
Environment-aware adaptive (used)	1,073	**0.0 %**	74.5 %	6.1 %

Only the adaptive method never mixes environments, while still cutting mostly at rest. Its 6.1 % of cuts during acceleration happen at environment changes, which the methodology requires the segmentation to respect.

Evidence

Environment	Segments	Good	Partial	Excluded	Usable hours	Median length
Urban	757	561	173	23	25.5 h	89 s / 0.50 km
Suburban	133	7	120	6	2.0 h	46 s / 0.30 km
Expressway	161	42	118	1	16.3 h	315 s / 6.2 km

Verification 11/11 PASS: every second with speed in exactly one segment, one environment per segment, no segment across a gap, chronological 1 s steps, distance preserved within 0.1 %, no segment > 15 min.

Limitation: Suburban appears as short patches along Galle Road, so its segments are short (median 46 s) and mostly partial. Intercity and Rural have no data.

Stage 7 – Driving feature extraction

Describe every usable segment with numbers – how fast, how much acceleration and braking, how often it stops, how far it goes – so segments can be compared and grouped.

What the code does

- Acceleration by central difference of 1 Hz speed. Each moving second is accelerating ($a \geq +0.1$ m/s²), decelerating (≤ -0.1) or cruising; stopped seconds are idle.
- **36 core features**: speed (mean, moving mean, median, max, SD, percentiles), acceleration (mean, SD, mean positive / negative, maxima, percentiles, RPA, PKE), time (duration, idle / accel / decel / cruise shares, stops, stops per km and per minute), distance and distance per speed band.
- **7 optional EV features** (energy, kWh/100 km, regeneration, power, SoC change) only when battery data is ≥ 90 % complete – not used for cycle construction.
- Feature-quality report: missing values, ranges, constant features, redundant pairs ($|r| \geq 0.9$).

Why

- These are the feature groups required by the methodology and standard in driving-cycle research (RPA and PKE measure how dynamic the driving is).
- The same definitions are reused in Stages 10 and 11, so the cycle and real driving are measured identically.

Evidence

Median per segment	Urban	Suburban	Expressway
Mean speed	19.3 km/h	25.1 km/h	68.7 km/h
Time stopped / cruising	13.5 % / 27.6 %	16.7 % / 20.6 %	0 % / 64.2 %
Stops per km	1.9	2.5	0
RPA	0.16 m/s²	0.20 m/s²	0.06 m/s²

1,021 segments, no missing core features, 12/12 checks PASS (incl. an independent re-computation of 20 random segments and the battery power sign convention). Found and passed on to Stage 8: 5 redundant pairs (e.g. RPA–PKE r = 0.9996) and unstable stops per km on very short segments (up to 64/km).

Stage 8 – K-Means clustering

Inside each environment, find groups of segments that are driven in a similar way (e.g. congested vs flowing). K-Means groups feature vectors; it does not build a cycle.

What the code does

- **Feature selection:** start from the Stage 7 candidates; stops per km → stops per minute; drop decel share (shares add up to 1); drop a feature only if it duplicates another ($|r| \geq 0.9$) in **every** environment. Identifiers, labels, times and coordinates are never used. Result: 14 features.
- Log-transform duration and distance, scale to mean 0 / SD 1, clip at ±3 SD (limits outlier pull).
- For k = 2…8: inertia (elbow), silhouette, Davies–Bouldin; choose the best silhouette among k whose every cluster has ≥ 5 % and ≥ 10 segments.
- Stability: 30 refits on 80 % subsamples + 5 random seeds (adjusted Rand index); outlier sensitivity without clipping.

Why per environment – evidence

One K-Means on all environments together found only “expressway cruising” vs “everything else” – all 734 Urban and 127 Suburban segments fell into one cluster. Clustering per environment is therefore needed to see the patterns inside each environment.

Evidence

Environment	k	Silhouette	Stability (ARI)	Clusters found (automatically)
Urban	2	0.33	0.96	congested stop-and-go (12.6 km/h, 36 % stopped) · flowing (22.5 km/h, 9 % stopped)
Suburban	3	0.39	0.98	mostly standing · flowing · stop-and-go
Expressway	3	0.55	1.00	toll plaza / queue (54 % stopped) · ramps (33 km/h) · cruising (81 km/h, 93 % of time)

Stable clusters (ARI ≥ 0.96) with weak-to-reasonable silhouettes mean the groups are real and repeatable but overlap at the edges – normal for driving, which changes gradually. Two different feature sets gave nearly the same clusters. Verification 7/7 PASS (incl. reproducibility).

Stage 9 – Representative segment selection

Choose a small set of real recorded segments that together behave like all driving in an environment. A cluster centre is an average of numbers, not a drive, so it is never used as a trace.

What the code does

- **Hold-out first:** trips (not segments) split 70/30 per route → 54 construction trips, 23 held-out trips kept for Stage 11.
- Each cluster gets a **time budget** = cycle length × its share of driving time (Urban 1,200 s, Suburban 600 s, Expressway 1,800 s).
- Two methods compared: **medoid ranking** (most typical segments) vs **coverage** (add the segment that brings the selection's statistics closest to the whole cluster). Good segments preferred; ≤ 2 per trip per cluster.
- Saves the list and the **unchanged** 1 Hz traces of the selected segments.

Evidence

Environment	Segments (trips)	Length	Mean speed	Idle share	Stops / km
Urban	11 (10)	1,215 s	−3.2 %	+1.5 %	+12.1 %
Suburban	11 (9)	578 s	+1.9 %	+1.3 %	+4.8 %
Expressway	4 (4)	1,698 s	+0.5 %	+6.4 %	−1.6 %

Coverage reproduced the environments better (mean error 0.037 vs 0.104 for medoids). Cluster time shares kept (e.g. Expressway 4.6 % toll / 2.9 % ramps / 92.5 % cruising). 9/9 checks PASS, including “no segment from a held-out trip”. Note: coverage optimises the statistics it is scored on, so the fair test is Stage 11.

Stage 10 – Driving-cycle construction

Join the selected real segments into one speed–time cycle per environment, without unrealistic jumps, and save each as a CSV of time (s) and speed (km/h).

What the code does

- Orders segments to minimise speed jumps at the joins (local search with swaps and moves; ties keep a fixed random order, mixing congested and flowing pieces).
- Joins directly when the jump ≤ 3.6 km/h (≤ 1 m/s² for one second); otherwise inserts a marked bridge at 1 m/s². Every cycle starts and ends at rest.
- Recorded segments are never smoothed, scaled or cut.

Evidence

Cycle	Length	Distance	Mean / max speed	Idle	Real segments	Added bridge
Urban	1,216 s	6.9 km	20.4 / 50.0 km/h	13.9 %	11	1 s (0.08 %)
Suburban	593 s	3.7 km	22.6 / 54.8 km/h	16.7 %	11	15 s (2.5 %)
Expressway	1,699 s	36.3 km	76.8 / 97.2 km/h	3.2 %	4	1 s (0.06 %)

> 97 % of every cycle is unchanged recorded driving. Against construction driving: mean speed, idle, cruise and RPA within ±6 % (Urban stops/km +12 %). 10/10 checks PASS: 1 s steps, start/end at rest, |a| ≤ 5 m/s², no join jump > 3.6 km/h, every segment used once and unchanged, saved CSV = (time_s, speed_kmh).

Stage 11 – Statistical validation

The independent test: are the cycles representative of the 23 held-out trips that were never used to build them?

What the code does

- Computes the same statistics for each cycle and for all held-out driving in that environment.
- **95 % confidence intervals** by resampling held-out *trips* 1,000 times (seconds within a trip are not independent). **Natural variation:** |cycle − held-out| ≤ 1 trip-to-trip SD. Rules fixed before seeing results.
- Distributions: speed bands, acceleration, and the speed–acceleration frequency distribution (SAFD). No p-values: with thousands of dependent seconds they would be meaningless; effect sizes are reported.
- Baselines: a single real trip vs the other held-out trips, and 200 random cycles of the same length.

Evidence

Cycle	Statistics passing	vs single real trip (mean |z|)	Better than random cycles	SAFD difference
Expressway	10 / 12	0.56 vs 0.66 ✓	95 %	21 % (trip 33 %, random 38 %) ✓
Suburban	11 / 12	0.12 vs 0.90 ✓	99.5 %	44 % (random 44 %)
Urban	11 / 12	0.54 vs 0.81 ✓	83 %	40 % (random 37 %) ✗

All three cycles are closer to the held-out driving than a typical single real trip and better than most random cycles – evidence that clustering and selection add value. 6/6 verification checks PASS (independence, same definitions as Stage 10, correct pooling).

Weaknesses reported honestly: the Urban cycle is slower and stops more than held-out urban driving (mean speed −14 %, idle +34 %, stops/km +39 %; inside the CI only because urban trips vary widely) and its SAFD is not better than random – the selection matched summary statistics, not distributions. Expressway has less accelerating/braking time (−20 % / −17 %). The Suburban maximum speed exceeds every held-out trip. Intercity and Rural cannot be validated.

Overall limitations and next steps

- **Synthetic data.** All results demonstrate the method only. Re-run Stages 1–11 on real EV trips before drawing conclusions about Sri Lankan driving.
- **Coverage.** Only urban (Galle Road) and expressway routes: no Intercity or Rural cycles.
- **Label verification.** Stage 5 labels are rule-based and not yet checked against ground truth or hand-checked reference points.
- **Thresholds.** Junction-density thresholds, stop definition and cycle lengths are reasoned choices; cite literature and recalibrate on real data.
- **Temporal resolution.** OBD speed every 3.7 s → 73 % interpolated → smoothed accelerations.
- **Minor leakage.** Stage 8 clusters were defined on all trips; selection and cycles use construction trips only.
- **Possible improvement.** Add the SAFD to the Stage 9 selection objective to improve the Urban cycle's speed–acceleration pattern.

Reference

Newson, P. and Krumm, J. (2009). Hidden Markov map matching through noise and sparseness. *Proc. 17th ACM SIGSPATIAL GIS*, 336–343. · Kaufman, L. and Rousseeuw, P. J. (1990). *Finding Groups in Data*. Wiley. · OpenStreetMap contributors (ODbL), accessed via OSMnx. Other thresholds should be supported with driving-cycle literature chosen and checked by the author.